1. Kérdés

*Milyen kapcsolat van a számítógép központi egységének (**CPU**) és az operációs rendszer fejlődése között.*

- Szoros, az operációs rendszer legjobb "barátja" a CPU!
- Semmilyen kapcsolat nincs, hiszen függetlenek az operációs rendszerek és a CPU gyártók egymástól.
- Elég laza, mert az operációs rendszer a memóriában helyezkedik el.
- Az operációs rendszer független a CPU-tól, hiszen emiatt telepíthetünk akár Windows akár Linux rendszert is a számítógépünkre.

2. Kérdés

*Mi a **virtuális rendszerek** mai széleskörű terjedésének, használatának az oka?*

- Az, hogy ma már ingyenesek ezek a rendszerek.
- Azért, mert az IBM, akitől származik a virtualizáció gondolata, nem kér licensz díjat, ugye ezért is ingyenes!
- Az erőforrások, elsősorban a CPU nyújtotta támogatások lehetősége.
- Az, hogy ma szükségem van több operációs rendszerre, és így tudom legkényelmesebben ezt biztosítani.

3. Kérdés

*Az **operációs rendszer**ek és a hardverek kapcsolatában egyre nagyobb szerepe van a **Firmware**-nek. Tényleg így van, mi ez?*

- Igen, az operációs rendszer részévé vált ez az újabb szoftver réteg is.
- Igen, a klasszikus külvilággal folytatott adatcsere (írás-olvasás) kérésre ma már jellemzően nem közvetlenül az eszköz (például merevlemez) válaszol, hanem egy beépített program.
- Ez csak egy elnevezés, a merevlemez ugyanúgy csatlakozik ma a rendszerhez, mint régen.
- Az operációs rendszerek réteges felépítésűek. Ez a hatás tükröződik vissza, a Firmware megjelenésében is, ez a kernel legújabb rétege!

4. Kérdés

*A **háttértár**on elhelyezkedő alkalmazások, programoknak és a memóriában futó folyamatok között van-e különbség?*

- Nincs különbség, ez csak szómágia! Igen.

- Ahogy egy úszó a medence partján partravetett hal, úgy a program a zsebben semmi!
- Igen, de csak elnevezésbeli a különbség, ahogy például a Pistát néha Pityunak vagy akár Istvánnak is lehet nevezni!
- Van különbség, de csak monoprogramozási környezetben!

5. Kérdés

*Ki biztosítja a **megszakítás**ok lehetőségét egy számítógépes környezetben?*

- Az időzítő, hiszen ő az aki akár az NMI vonalat is el tudja érni!
- A szoftveres megszakítás, az operációs rendszer hathatós támogatásával.
- Az operációs rendszer, külön (I/O) kapuja van ezen kérések kezelésére.
- A **CPU**, külön lába, kapuja van ezen kérések kezelésére.

6. Kérdés

*Még ma is a legfontosabb **háttértár**unk a **mágneses merevlemez**. Mit jelent az, hogy a lemezen definiált sávok-szektorok metszésében elhelyezkedő blokk mérete 512 bájt?*

- Meghatározza lemezfoglaltsági térkép szerepét, ha ez van, akkor nem kell ilyen.
- Ez nem jelent semmit, a klaszter számít.
- Ez az operációs rendszertől függ, tehát más is lehet.
- Azt, hogy a legkisebb lemezfoglalási egység ennyi lehet.

7. Kérdés

*Mi a **lemezpartíció**?*

- Azon lemezrész, amit virtuális gépként használhatunk.
- Egy merevlemezen logikai lemezrész kialakítása.
- A C:, D: lemezegységek a partíciók.
- Azon lemezrész, ami csak swap funkciót biztosít.

8. Kérdés

*Az **Input-Output** eszközök között miért kitüntetett szerepű a diszk, a háttértár?*

- Mert ezen van mindenünk.
- Mert a szalagegység lassú és már nem létezik.
- Már nem kitüntetett, mert félvezető(flash) elvű tárolást használunk.

- Mert ez a legmegbízhatóbb.

9. Kérdés

*Mi a szerepe a **lemezkérés** ütemezéseknek?*

- A lemez forgási sebességét csökkenteni vagy növelni a hatékonyság, hozzáférési idő csökkentésében.
- Optimális sávszélesség elérése a háttértárak felé.
- Az író/olvasó fej optimális működését biztosítani.
- Az interleave helyes beállítása.

10. Kérdés

*Milyen kötelező kiegészítő **információk**at, **attribútumok**at kell tárolni egy **fájl**ról vagy **könyvtár**ról?*

- Van ilyen, az IEEE2020 szabvány definiálja.
- Van ilyen ISO szabvány.
- Nincs ilyen, emiatt lehet hidden fájl vagy hidden fájlrenszer is.
- Nincs ilyen általános előírás, de minden fájlrendszer típus pontosan előírja, hogy hol, hogyan és milyen információkat tárol az adott bejegyzésről.

11. Kérdés

*Mi az **i-node** (index-node) tábla?*

- A Unix stílusú operációs rendszerek fájlrendszerének alapstruktúrája, táblája. Egy node tartalmazza a fájl attribútumokat és a fájl blokkok helyét a lemezen.
- A Windows operációs rendszerek fájlrendszerének alapstruktúrája, táblája. Egy node tartalmazza a fájl attribútumokat és a fájl blokkok helyét a lemezen.
- Csak a legújabb Linux stílusú operációs rendszerek fájlrendszerének alapstruktúrája, táblája. Egy node tartalmazza a fájl attribútumokat és a fájl blokkok helyét a lemezen.
- Egy általános index tábla, ami a fájlrendszer indexelését tartalmazza.

12. Kérdés

*Mi a **naplózott fájlrendszer**?*

- Az adatbázis rendszerek mintájára, a nagyobb megbízhatóság miatt, naplózza (log) a végzett műveleteket, így fájlrendszer hiba esetén ezen naplóból visszaállítja a rendszer egységét.

- A mai fájlrendszerek már nem ilyenek, a naplózás azt jelenti, hogy mint egy jelenléti naplót kitöltünk, ezzel regisztráljuk a fájlok jelenlétét.
- A naplózott fájlrendszer naplót vezet a fájlok használati gyakoriságáról, majd az LRU algoritmus segítségével cserél ha szükséges.
- A naplózott fájlrendszer naplót vezet a fájlok használati gyakoriságáról, ezzel biztosítva egy gyorsabb működést.

13. Kérdés

*Mit értünk a **folyamatok szekvenciális modell**je alatt?*

- MultiCore környezetben az erőforrásokat adott időközönként, a folyamatok rendelkezésére bocsátjuk. Ha lejár egy folyamat ideje, nem váltunk a következőre, hanem megvárjuk a folyamat végét.
- Multitask környezetben az erőforrásokat adott időközönként, a folyamatok rendelkezésére bocsátjuk. Ha lejár egy folyamat ideje, nem váltunk a következőre, hanem megvárjuk a folyamat végét.
- Multitask környezetben a CPU+Memória+I/O eszközöket adott időközönként, a folyamatok rendelkezésére bocsátjuk. Ha lejár egy folyamat ideje, váltunk a következőre, ezzel elérve a párhuzamos végrehajtás hatását (multiprogramozás).
- Monoprogramozási környezetben kiosztjuk a meglévő erőforrásokat. A folyamatok közti váltás a multiprogramozás.

14. Kérdés

*Hogyan biztosítja a **szemafor** segítségével a jó kölcsönös kizárást az operációs rendszert?*

- Azzal biztosítja a jó kölcsönös kizárást, hogy ezen a "változón" végzett művelet oszthatatlan, nem lehet megszakítani.
- Azzal biztosítja a jó kölcsönös kizárást, hogy ezen a "változón" végzett művelet felhasználó szinten valósul meg.
- Posix ajánlás szerint a "volatile"jelzővel biztosítjuk a jó kölcsönös kizárást.
- Azzal biztosítja a jó kölcsönös kizárást, hogy ezen a "változó" nem igényel közös memóriát.

15. Kérdés

***Interaktív, multiprocessz rendszerek**ben melyik rendszer modell az, ami az egyik legkorábbi, de ma újra "másodvirágzását" éli?*

- Monoprogramozási modell
- Preemptív modell
- Kooperatív modell
- Környezetváltásos modell

16. Kérdés

*Mi a **folyamatok** közti kölcsönös kizárás fogalma?*

- A monoprogramozás világának folyamat kizárási módszere.
- Módszer ami engedi, hogy a közös adatokat egyszerre több(multitask) folyamat tudja használni.
- Módszer ami biztosítja, hogy a közös adatokat egyszerre annyi folyamat tudja használni, amennyit beállítunk.
- Módszer ami biztosítja, hogy a közös adatokat egyszerre csak egy folyamat tudja használni.

17. Kérdés

*Mi a problémája a tevékeny várakozással megvalósított **kölcsönös kizárás**nak?(Pl, **Peterson algoritmus**)*

- Ha erőforrás szűkében vagyunk, akkor ez rossz, mert CPU időt pazarol.
- Ez a megoldás ideális, nem véletlen, hogy a mai rendszerek "SpinLock" néven is használnak hasonlót.
- Ha erőforrás szűkében vagyunk, akkor ez rossz, mert memóriát pazarol.
- Ez a megoldás nem jó, mert többen is kritikus szekcióba kerülhetnek

18. Kérdés

*Mi nem jellemző a kötegelt rendszerek sorrendi (**FCFS-Fist Come First Service**) folyamat ütemezőjére?*

- Kell ismerni a futási időket előre.
- Nem kell ismerni a futási időket előre.
- A lista első helyén lévő folyamat következik, ami addig fut amíg nem végez, vagy nem blokkolódik, pl I/O esetén.
- Pártatlan, láncolt listában tároljuk a folyamatokat.

19. Kérdés

*Mi az **üzenetküldés**sel biztosított kölcsönös kizárás fő előnye?*

- Az, hogy a hálózati kommunikációhoz hasonló elvet követ.
- Még a monitornál is könnyebb használat.
- Nem szükséges közös (rendszer) memória a helyes kölcsönös kizárás biztosításához.
- Közös (rendszer) memória esetén nincs lehetőség a helyes kölcsönös kizárás biztosítására.

20. Kérdés

*Mi a **kölcsönös kizárás**t biztosító monitor használatának a korlátja, ha van ilyen?*

- Közös memória hiányában nem használható.
- Külön magon futó folyamatok esetén sose használható.
- Azonos magon futó folyamatok esetén nem használható.
- Közös memória hiányában is jól használható.

21. Kérdés

*Mi a **prioritásos ütemező**k alapelve és az ebből következő probléma?*

- Mindig az alacsonyabb prioritású folyamat következik, és így megakadályozzuk a kiéheztetést.
- Az alapelv, ha van magasabb prioritású folyamat ami CPU-ra vár, azt fogja választani az ütemező, így az alacsony prioritású folyamatra a kiéheztetés veszélye fenyeget.
- Mindig a magasabb prioritású folyamat következik, ami biztosítja a helyes működést.
- Felváltva választunk a magas majd az alacsony prioritási osztályból, így nincs probléma.

22. Kérdés

*A mai **Windows** rendszerekben használt **prioritásos ütemező**je milyen módszert használ a **kiéheztetés** elkerülésére?*

- A Windows az Arányos ütemező elvét követve kerüli el a kiéheztetést.
- A Windows a strucc algoritmust használva kerüli el a kiéheztetést.
- Az alacsony prioritású folyamatok **egy idő után** magas prioritási osztályba kerülnek, majd ott CPU időt kapnak, az idő letelte után vagy terminálnak vagy **maradnak** az eredeti osztályban.

- Az alacsony prioritású folyamatok **ideiglenesen** magas prioritási osztályba kerülnek, majd ott CPU időt kapnak, az idő letelte után vagy terminálnak vagy **visszakerülnek** az eredeti osztályba.

23. Kérdés

*Mi a **virtuális memória** kezelés?*

- A SWAP partíció használatával megvalósuló memória kezelés.
- Virtuális gépen végzett memória kezelés.
- Nincs ilyen, hiszen egy CPU csak a fizikai memóriában lévő kódot tudja végrehajtani.
- Mikor a létező fizikai memóriától nagyobb memóriát is engedélyez használni az operációs rendszer.

24. Kérdés

*Mi a **memóriakezelő**?*

- Olyan felhasználói szintű program, ami az IDT kiosztásáért felel.
- Olyan általában kernel szinten elhelyezkedő program, ami a dinamikus memória igények, foglalások, felszabadítások során jelentkezik és azokat megoldja.
- Olyan kernel szintű program, ami az operációs rendszer rögzített memóriát használó részeit kezeli.
- Olyan felhasználói szintű program, ami a felhasználói alkalmazások memória igényeit teljesíti.

25. Kérdés

*Melyik feladat nem tartozik a memória kezelő (**Memory Management Unit, MMU**) hatáskörébe viruális memóriakezelés során?*

- A program betöltésekor szükséges LDT, GDT táblák módosítása.
- A virtuális laptábla kezelése.
- A Copy On Write (COW) beállítása.
- A Data Execution Prevention (DEP) beállítása.

26. Kérdés

*Mikor nem teljes folyamat mozgatása történik a **memória** és **háttértár** között?*

- Soha nincs és nem is volt ilyen.
- Virtuális memória kezelés esetén, csak extrém esetben lehet ilyen.
- Multiprogramozás rögzített memória szeletek használata esetén.
- Multiprogramozás memória csere használata esetén.

27. Kérdés

*Mi a probléma figyelmen kívül hagyásának (**strucc algoritmus**) lényege?*

- Nem foglalkozunk az eszközök felé irányuló kérések ütemezésével, csak biztosítjuk a kérések küldését.
- Nem foglalkozunk az eszközök felé irányuló kérések ütemezésével, a kérések küldését az alkalmazás közvetlenül végzi az eszközhöz.
- Ütemezzük a kéréseket a CDF (Closest Device First) algoritmus szerint.
- Ütemezzük a kéréseket a NOOP (No Operation) szerint.

28. Kérdés

*Melyik a **legnagyobb prioritás**ú megszakítás kérés?*

- Az 1-es sorszámú megszakítás.
- Az NMI, a nem maszkolható interrupt.
- A 15-ös sorszámú megszakítás.
- A szoftveres megszakítás.

29. Kérdés

*A **bankár algoritmus**ban mit nevezünk biztonságos állapotnak?*

- Azt amiből már nem tudjuk befolyásolni a folyamatok befejezését a helyes erőforrás (hitel) kihelyezéssel.
- Az amikor egy folyamat rendben terminál.
- Azt amiből még mi tudjuk befolyásolni a folyamatok befejezését a helyes erőforrás (hitel) kihelyezéssel.
- Azt amiből még mi tudjuk befolyásolni a folyamatok befejezését a helyes erőforrás (hitel) visszakövetelésével.

30. Kérdés

*Mire jó lehetőség a **folyamatok-eszközök gráfmodell**jének a használata?*

- A probléma figyelmen kívül hagyására.
- A holtpont felismerésére és kiküszöbölésére.
- Az éhenhalás elkerülésére.
- A holtpont dinamikus elkerülésére

31. Kérdés

*A számítógépek fejlődése során megkülönböztetünk számítógépes, **operációs rendszer generációk**at. Mi az ami leginkább befolyásolta ezen generációkat, azok kialakulását?*

- A CPU kernel-user szintjének megjelenése.

- Sokféle szempont alapján lehet rendszereket megkülönböztetni, de leginkább a Single-Task, Multi-Task rendszerek fejlődése.
- A külvilág igénye. Például az első generációknál leginkább számításigényes megoldásokra volt szükség, így azt támogatták.
- Sokféle szempont alapján lehet rendszereket megkülönböztetni, de leginkább az elektronikai támogatás fejlődése.

32. Kérdés

*A **perifériák**kal végzett kommunikációra melyik alábbi állítás nem érvényes?*

- Folyamatosan érdeklődünk, hogy például elkezdődött-e a mérkőzés?
- Akár speciális eszközt is biztosíthatunk nagy mennyiségű adat(memória) eléréshez.
- Az eszközök adatai fájlban gyűlnek, így az operációs rendszer bináris fájl műveleteket biztosít ehhez!
- Berúgjuk az ajtót, megszakítjuk a futó dolgokat és jelezzük, "meccsidő" van!

33. Kérdés

*Mit biztosít, mit jelent, hogy a **CPU üzemmód**ok bővültek, és megjelent például a "kernel" és "user" mód?*

- Ez a lehetőség azt jelenti, hogy az operációs rendszer tud a hatalmával élni és **"teljhatalmú úr"** a folyamatok fölött.
- Olyan lehetőséget ad, amivel mi is készíthetünk Operációs rendszert!
- Olyan lehetőséget ad, amivel jobban igazítható egy-egy alkalmazástípushoz a rendszerünk.
- Ez a lehetőség azt jelenti, hogy az operációs rendszer nem tud a hatalmával élni és nem lehet "önkényúr" a folyamatok fölött.

34. Kérdés

*Melyik **lemez ütemezési algoritmus**t célszerű használni SSD háttértárak esetén?*

- Az SSTF algoritmust.
- A C-SCAN algoritmust.
- A SCAN algoritmust.
- Az FCFS algoritmust (ssd-ben nincs fejmozgás minden azonnal elérhető, ezért valszeg a kérés beérkezés sorrendje a legjobb)

35. Kérdés

*Mit jelent a **háttértárak** megbízhatósága?*

- Egy módszer, ami a sávszélességet növeli.
- Egy módszert, amivel a tárolási redundanciát növelni lehet.
- Egy módszer, ami a hatékonyságot növeli.
- Módszer ami biztosítja, hogy a közös adatokat egyszerre csak egy folyamat tudja használni.

36. Kérdés

*Milyen szerepe van ma a **mágnesszalagos háttértár**nak?*

- Mivel nagyon drága, ezért már nem használják.
- Mivel lineáris adatelérést biztosít, végig kell tekerni a szalagot a kívánt állományig, lassú, ezért nem használt.
- Nagygépes környezetek elengedhetetlen perifériája az LTO6-os egység.
- Lecsökkent a szerepe, nagy mennyiségű adat, megbízható tárolását biztosítja.

37. Kérdés

*Egy **háttértár**on hány különböző fájlrendszert használhatunk egyidejűleg?*

- Ahány egységre, partícióra bontjuk a háttértárat.
- Ahány fájlrendszert az operációs rendszer ismer.
- Kettőt.
- Csak egyet.

38. Kérdés

*A **preemptiv** operációs rendszer modell "mit előz meg, kitől kit véd meg"?*

- A határidők megjelenése arra ösztönzi az alkalmazásokat, hogy gyorsan fejezzék be a válaszadásokat, így biztosítva a rendszert.
- A modellben bizonyos idő után megkérik az alkalmazást, hogy mondjon le a CPU-ról, így megelőzi a rendszer kisajátítását, összeomlását.
- A **modellben** bizonyos idő után **elveszi a rendszer** a vezérlést az alkalmazástól, így megelőzi a rendszer kisajátítását, összeomlását.
- Nem előz meg semmit, hiszen a mai rendszerekben is találkozunk rendszerhibákkal.

39. Kérdés

*A **Peterson féle** kölcsönös kizárás használatánál mi a probléma az alábbi kódrészlet piros sorai felcserélése során?*

```
void belepes(int proc)
{
	int masik;
	masik=1-proc; //mivel N=2…
	akarja[proc]=1; //processz futni akar
	kovetkezo=proc;
	while( kovetkezo==proc && akarja[masik]);
}
```

- Ha bármikor vált az ütemező "éhen nem halhatnak a filozófusok", nincs probléma.
- Ha rosszkor vált az ütemező "egymásnak eshetnek a filozófusok", többen is kritikus szakaszba kerülhetnek.
- Ha a sorok előtt vált az ütemező az problémát okozhat.
- Ha a sorok után vált az ütemező, az okozhat problémát.

40. Kérdés

*Miért biztosít az operációs rendszer **IPC** (Inter Process Communication) lehetőséget a folyamat számára?*

- Azért, hogy a kernel-user szint meg tudjon valósulni.
- Azért mert a POSIX szabvány ezt is előírja!
- Azért, hogy normális rendszer legyen.
- Azért, hogy el tudjuk zárni az I/O eszközöket a felhasználóktól.

41. Kérdés

*A **szekvenciális folyamatmodell** alapján melyik állapotban lehet csak egy folyamat?*

- Futásra kész állapotban, mert így lesz egyértelmű, hogy ki lesz a következő "futó".
- Blokkolt állapotban, mert egy blokkolt folyamat a következőt is blokkolná ha ebbe az állapotba szeretne bekerülni.
- Futó állapotban.
- Mindegyikbe lehet egy folyamat, ha nincs sok futó alkalmazás a rendszerben.

42. Kérdés

*Mi a **szemafo**r? (informatikában használt)*

- Olyan speciális változó, aminek segítségével alvás-ébredés alapú kölcsönös kizárást tudunk biztosítani.

- Olyan felhasználói szintű objektum, aminek segítségével alvás-ébredés alapú kölcsönös kizárást tudunk biztosítani.
- A felhasználói programok által foglalt összes memóriát ezzel a speciális változóval lehet kezelni!
- Olyan speciális változó, amit csak a kernel használ automatikusan az alvásébredés alapú kölcsönös kizárás megvalósítására.

43. Kérdés

*Mivel javítják az alapértelmezett **prioritásos ütemezési elv**et, hogy elkerüljék a kiéheztetést?*

- Valamilyen prioritási osztályok közti átmenetek megvalósításával.
- Nem javítják semmivel.
- Azzal, hogy minden felhasználói folyamat 0-ás prioritással fut.
- Azzal, hogy egy prioritási osztályban Round-Robin ütemezést valósítunk meg.

44. Kérdés

*A **Fogyasztó-Vásárló probléma** megoldásában, miért probléma ha felcseréljük a két pirossal jelölt sort?*

```
void pék()
{
int kenyér;
while (1)
 { kenyér=pék_süt();
down(&üres);
down(&szabad);
kenyér_polcra(kenyér);
up(&szabad);
up(&tele);
 }
}
```

- Mert ha az ütemező a
  kenyér_polcra(kenyér);
  utasítás előtt vált, holtpont alakulhat ki.
- Mert a kölcsönös kizárás úgy hibás lehet.
- Nem probléma ha felcseréljük a két sort.
- Mert ha az ütemező a

kenyér=pék_süt();
utasítás után vált, holtpont alakulhat ki.

45. Kérdés

*Jelölje meg mi nem a **Sorsjáték** ütemezés jellemzője?*

- Prioritásos ütemezés, ami azt jelenti, hogy ennek megfelelően osztja szét a folyamatok között a sorsjegyeket.
- Sorsjáték szerűen a kiosztott jegyek alapján választunk egy folyamatot.
- Hasonlít a Garantált ütemezéshez, csak a CPU idő számolás más elvek alapján történik.
- Az időintervallum lejártakor egy új sorsjegyet kap a folyamat és legközelebb rövidebb ideig futhat.

46. Kérdés

***Virtuális memóriakezelés** során mi a címfordítás?*

- A programok a **virtuális térben** helyezkednek el, így a **virtuális címből fizikai címet** kell előállítani.
- A programok a virtuális térben helyezkednek el és az első szabad fizikai címhez rendelést hívjuk címfordításnak.
- A programok a virtuális térben helyezkednek el, így annak megadása, hogy melyik lapon van az aktuális kódrész.
- A program a fizikai memóriában fut, és ebből a címből meg kell határozni a virtuális címet.

47. Kérdés

*Melyik feladat nem tartozik a memória kezelő (Memory Management Unit, **MMU**) hatáskörébe viruális memóriakezelés során?*

- A Data Execution Prevention (DEP) beállítása.
- A Copy On Write (COW) beállítása.
- A virtuális laptábla kezelése.
- A program betöltésekor szükséges LDT, GDT táblák módosítása.

48. Kérdés

*Mi a **megszakítás**(interrupt) fogalma?*

- A megszakítás kérés jellemzően I/O eszközökhöz kötődik, egy csapda, amivel azt jelzi, hogy kernel módba kell váltani!
- A megszakítás kérés nem kötődik I/O eszközökhöz, egyszerűen egy gépi kódú utasítás.

- A megszakítás kérés jellemzően I/O eszközökhöz kötődik, egy időzítő jelzés küldése a CPU felé, amiben azt jelzi, hogy lejárt az adatérvényesség ideje.
- A megszakítás kérés jellemzően I/O eszközökhöz kötődik, egy **valós jelzés küldése** a CPU felé, ami azt jelzi, hogy szeretné az eszköz ha vele foglalkozna a **CPU!**

49. Kérdés

*Mi az **Input-Output** eszközök feladata?*

- Nincs is feltétlenül szükség I/O eszközökre egy számítógép működéshez!
- Megbízhatóan ütemezze az adatok fogadását és küldését.
- A számítógép és külvilág kapcsolatának a biztosítása.
- Mivel a számítógép digitális eszköz, ezért az analóg-digitális jelátalakítás a feladata.

50. Kérdés

*Melyik tulajdonság nem a **Round-Robin** ütemező jellemzője?*

- Egyszerű prioritásos ütemező interaktív rendszerekben.
- Körkörös listában nyilvántartott folyamatok közül mindig a következőt
- választjuk.
- Mindenkinek azonos időszelet jut.
- Egyszerű, pártalan ütemező interaktív rendszerekben.

51. Kérdés

*Melyik **program réteg** nem tartozik az eszköz és az **operációs rendszer kapcsolatai** közé?*

- Eszköz függő operációs rendszer interfész.
- Felhasználói I/O könyvtár.
- Megszakításkezelés rétege.
- Eszköz független operációs rendszer interfész.

52. Kérdés

*Mit nevezünk **Operációs rendszer**nek? Válassza ki a nem megfelelő választ!*

- Olyan program ami egyszerű felhasználói felületet nyújt, eltakarva a számítógép(rendszer) eszközeit.
- Azt a programot ami egy megoldást biztosít egy feladathoz! Például ilyen a mosógépet működtető beépített program.

- Az Operációs Rendszer tekinthető egy erőforrás menedzsernek is, ami a felhasználó akaratának megfelelően kezeli az erőforrásokat.
- Az Operációs Rendszer tekinthető egy kiterjesztett (virtuális) gépnek, ami a felhasználó kívánságait teljesíti.

53. Kérdés

*Sok elvárásunk van egy **operációs rendszer** felé, ilyen a hatékonyság, megbízhatóság, rendelkezésre állás, kompatibilitás, alacsony energia felhasználás. Melyik alábbi válasz nem segíti a nagyobb rendelkezésre állást?*

- Használok szerver "clustert".
- Paritás bitjavítású memóriát (ECC) használok.
- Paritás elvű háttértár redundanciát használok.
- Nem használok szerver "clustert".

54. Kérdés

*Mi a szerepe a mai **Operációs rendszerek**nek a felhasználói alkalmazások felé?*

- Az alkalmazás szabad, ha kell közvetlenül is azt csinál amit akar.
- A mai rendszerekben bár eltakarja a hardver eszközöket az alkalmazásoktól de tetszőleges elérésünk lehet az eszközök vagy bármilyen szolgáltatás felé.
- A Single-Task világ szabadságát nyújtja, hiszen nem lehet a jogokat korlátozni.
- A mai rendszerekben eltakarja a hardver eszközöket az alkalmazásoktól és csak a kínált (eszköz)lehetőséggel élhet egy alkalmazásunk.

55. Kérdés

*Melyik megoldás ad helyes megoldást a **jó kölcsönös kizárás**ra az alábbiak közül?*

- Megszakítások tiltása
- TLS alapú belépés, kilépés függvény alapú megoldás.
- Szigorú váltogatás.
- Osztott vagy zárolás változó (0,1 értékek) használata.

56. Kérdés

*Melyik nem használt **RAID** technológia?*

- RAID0+1

- RAID1+0
- RAID0
- RAID2+0

57. Kérdés

*Mi nem jellemzője a **RAID 5** tömbnek?*

- Dedikált paritásdiszk használat.
- Redundáns adattárolást biztosít.
- Legalább 3 lemez kell a tömbbe.
- 1 lemez meghibásodása esetén nyújt biztonságot.

58. Kérdés

*Mit nevezünk **VFS**(Virtual FileSystem) fájlrendszernek?*

- A virtuális fájlrendszer egy újgenerációs fájlrendszer, ma még csak a legújabb rendszerek támogatják.
- A virtuális gépek értelemszerűen csak virtuális fájlrendszert tudnak használni, így az ebben a környezetben elérhető fájlrendszerek azok.
- Nincs virtuális fájlrendszer, csak valós.
- Az operációs rendszerek általános (virtuális) fájlrendszer réteget használnak, ami egységes elérést biztosít a felhasználók és a különböző fizikai fájlrendszerek között.

59. Kérdés

*Mi nem jellemzi a **FAT** fájlrendszert?*

- A FAT rendszereknél szükséges a töredezettség mentesítés (hagyományos merevlemezek esetén).
- A FAT, ahogy a nevében is van (T-Table), egy táblázat alapú fájlrendszer.
- Egy ma is használt láncolt listás tárolási elvű fájlrendszer.
- Elég régi rendszer, emiatt a nagyobb lemezek esetén nem elég hatékony.

60. Kérdés

*Mi az előnye és egyben a hátránya a **szemafor** használatnak a tevékeny várakozással szemben?*

- Az alvás - ébredés elv használata, ami sajnos időigényes teljes környezetváltással (Context switch) jár.
- Nincs ilyen, csak előnye van az alvás-ébredésnek.

- Az alvás - ébredés elv használata, ami sajnos hosszú lokális környezetváltással (Local switch) jár.
- Nincs ilyen, mert az alvás-ébredést kombinálja az ébredés-alvással.

61. Kérdés

*A folyamatok kapcsolatában mit mutat meg a "**folyamatfa**"?*

- A folyamatfa a folyamatok "rokonságát" tudja megmutatni.
- A folyamatfa a reinkarnációs szerverek megszüntetésében tud segíteni.
- A folyamatfa azokat a csomópontokat mutatja meg, amik megszüntetik a folyamat köröket.
- A folyamatfa a mono környezetek kapcsolati hálóját elemzi.

62. Kérdés

*Jelölje meg, melyik állítás nem igaz **szálak**ra!*

- Folyamatnak és szálnak is van saját regiszter készlete.
- A szálaknak külön fájlleírói vannak.
- A folyamat és a szála ugyanabban a memóriatérben helyezkedik el.
- Folyamatnak és a szálnak is van utasításszámlálója.

63. Kérdés

*A **folyamatok nyilvántartás**ának mi az elsődleges eszköze az **operációs rendszer**ben?*

- A száltáblázat, ami külön kiemeli a folyamatok és szálak kapcsolatát.
- A GDT, Global Descriptor Table, ugyanis ebben minden globális folyamatadat látszik.
- A laptábla, ami pontosan mutatja, hogy a folyamat milyen lapokat használ.
- A folyamattábla, ami minden folyamatkezeléshez szükséges információt tartalmaz.

64. Kérdés

*Jelölje meg mi nem jellemzője a **CFS** (Completely Fair Scheduler) ütemezőnek?*

- Alapértelmezett prioritásos ütemező Linux rendszerekben, Molnár Ingó volt ELTE hallgató nevéhez köthető az ütemezési elv.
- Prioritásos ütemező Linux környezetben, a kiéheztetés veszélye fennáll.
- A CPU idő "fair" kiosztási elve a Garantált ütemezéshez hasonlít.

- Olyan prioritásos ütemező, aminél bináris fában (RedBlackTree) tároljuk az elhasznált CPU időket.

65. Kérdés

*Mi nem jellemző a **BIOS**, **UEFI**(Unified Extensible Firmware Interface)mód tulajdonságaira?*

- 32 és 64 bites változat létezik csak.
- Nagy lemezméret (nagyobb mint 2TB) támogatása.
- Van MBR fizikailag, de helyette a GPT(Guid Partition Table) használható.
- Több mint 4 partíció használható.

66. Kérdés

*Melyik állítás nem igaz az **Író-Olvasó** problémára?*

- Az olvasók számlálójának elérését és az író kölcsönös kizárását az alábbi részlet biztosítja.
  down(mutex);
  Rc++;
  if (rc==1) down(database);
  up(mutex);
- Az írók számlálójának kölcsönös kizárását az alábbi részlet biztosítja.
  down(mutex);
  Rc++;
  if (rc==1) down(database);
  up(mutex);
- Egyszerre egy író lehet az adatbázisban, amit a database szemafor biztosít.
- Az alábbi részlet az író magányát biztosítja.
  down(database); // kritikus
  írunk_adatbázisba();
  up(database);

67. Kérdés

*Mi a **TLB** (Tranlation Lookaside Buffer) buffer?*

- A TLB jelentése Total Load Balancer, a számítógépes klaszterek világában használjuk.
- Címfordításnál az utolsó directory page táblának a neve.

- Címfordításnál a mai rendszerekben ebből a CPU-n belül lévő táblából nézi meg először az MMU, hogy a virtuális címnek mi a fizikai megfelelője.
- Címfordításnál a memóriakezelő az elsődleges laptáblát így is szokta nevezni.

68. Kérdés

*A mai **operációs rendszerek**ben melyik **lapcsere** algoritmus leginkább preferált?*

1. A Strucc algoritmus.
2. A FIFO algoritmus.
3. Az Óra lapcsere (Round Robin)
4. Az LRU (Least Recently Used) algoritmus.

69. Kérdés

*Mi a **laptáblák** kezelése során jelentkező jellemző probléma?*

- Jellemzően a laptábla mérete!
- Ha kisebb a virtuális memória mint a fizikai, az gond lehet.
- Az, hogy a mai környezetben nagy a fizikai memória.
- Az ha bájtonkénti memória nyilvántartást vezetünk.

70. Kérdés

*Mit nevezünk **többszintű laptábl**ának?*

- Azt amikor az egy laptábla helyett többet használunk.
- Amikor szükségünk van egyéb memória információkra (pl jogosítvány) is.
- Amikor a címzés előző részét (a szegmens mellett) egy másik táblában tároljuk.
- Nincs a laptáblának több szintje.

71. Kérdés

*Mi a **Bankár algoritmus**?*

- Az erőforrásgráf újratervezésével megadott stratégia.
- Felváltva használt bizonytalan-biztonságos állapotsorok kezelése.
- Mint a valós életben a bankár, kamatot számol fel egy erőforrás igénylés során.
- Az esetlegesen kialakuló holtpont dinamikus elkerülésének stratégiája.

72. Kérdés

*Milyen **Input-Output** eszközöket különböztet meg egy operációs rendszer?*

- Csak párhuzamos van, soros nincs, helyette az USB bukkant fel!
- Csak karakteres eszköz létezik, egy blokk sok karakterből áll.
- Időzítőket, blokkos és karakteres eszközöket.
- Csak blokkos eszköz létezik, a karakteres eszköz egy egybájtos blokkos eszköz.

73. Kérdés

*Mitől van az, hogy minden egyes **Linux** verzióban a névtelen cső létrehozás a "**pipe**" híváshoz kötődik?*

- Mert a cső kifejezés angolul a Pipe!
- Azért mert a Unix születésekor ezt definiálták!
- Mert mindenki követi az ajánlásokat! (Posix)
- Véletlen, akár a tube is lehetne!

74. Kérdés

*Milyen szerepe van az operációs rendszerek által nyújtott **API** (Application Programming Interface) könyvtáraknak?*

- Csak JVM vagy .NET esetén van szerepük, mert itt nem gépi kódra fordulnak az alkalmazások!
- Kulcsszerepe van, mert API nélkül nincs PAPI (Programmable API).
- Kulcsszerepe van, mert az API-k a programozók (Programozható API, PAPI :) legjobb barátai!
- Nincs nagy szerepük, hiszen gépi kódra fordulnak az alkalmazások.

75. Kérdés

*Melyik állítás nem igaz a **kölcsönös kizárás**t biztosító **monitor**ra?*

- A monitor eljárásban a condition "változókon" lévő wait, signal művelettel biztosítják, hogy ne blokkolódjanak a folyamatok.
- A monitor megvalósítás egy szemaforon alapuló biztonságosabb kölcsönös kizárást biztosító objektum.
- A monitor egy operációs rendszer nyújtotta egyszerűbb, biztonságosabb kölcsönös kizárást biztosító eszköz.
- A monitor a fejlesztő környezetek nyújtotta egyszerűbb, biztonságosabb kölcsönös kizárást biztosító eszköz.

76. Kérdés

*Mit nevezünk **laptábl**ának a **virtuális memória** kezelés során?*

- Azt a táblázatot, ami a folyamatok által foglalt szegmenseit tartalmazzák.
- Azt a táblázatot, amiben a memória cserét használó környezetben a memóriát nyilvántartjuk.
- A JVM által használt táblázatot.
- Azt a táblázatot, amiben nyilvántartjuk, hogy melyik virtuális lap hol helyezkedik el a fizikai memóriában.

77. Kérdés

*Milyen **memóriát** kezel az **operációs rendszerek** memória kezelője?*

- A felhasználói szintű programok által foglalt összes memóriát!
- A külső eszközök meghajtói és a felhasználói alkalmazások memóriáját.
- Csak a kernel memória foglalását.
- A felhasználói szintű programok által lefoglalt dinamikus memóriát.

78. Kérdés

*Mi a lényege a megelőzés **holtpont stratégia** követésének?*

- Legalább egy feltétel biztosítása.
- Nincs is ilyen stratégia.
- Legalább egy holtpont feltétel meghiusítása.
- A Coffman féle feltételek biztosítása.

79. Kérdés

*Mi a probléma a **bankár algoritmus**sal?*

- Csak egy erőforrásra létezik a bankár algoritmus.
- Nincs semmi probléma a bankár algoritmussal.
- Túl nagy árat kell fizetni (1:250) a bankár algoritmusért.
- Az, hogy túl sokat kellene tudni hozzá, hogy használni lehessen.

80. Kérdés

*Mi a **holtpont** (deadlock)?*

- Gyakorlatilag akkor jelentkezik, ha monoprogramozási környezetben vagyunk és eltűnik az eszköz a rendszerből.
- Két vagy több folyamat egy erőforrás megszerzése során olyan helyzetbe kerül, hogy egymást blokkolják a további végrehajtásban.
- Mikor egy folyamat egy erőforrás megszerzése során olyan helyzetbe kerül, hogy önmagát blokkolja a további végrehajtásban.
- Mikor nem jut CPU időhöz egy eszköz!

81. Kérdés

*Mi nem jellemző egy mai **4. generációs** (operációs) rendszerre?*

- Ebben csak Multi-Task, GUI a megengedett felhasználói felület.
- A Multi-Task rendszer mellett jelen van akár a Single -Task rendszervilág is.
- Nagyfokú integráció (LSI) jellemzi a komponenseket.
- Ma minden rendszer tartalmaz CPU-t.

82. Kérdés

*Mit nevezünk a **CPU** működési módok között megjelenő **Root** módnak vagy ezt gyakran **Ring -1** módnak is nevezik?*

- Ezzel a móddal tudjuk bekapcsolni a szoftveres virtualizációt.
- Ugye egy virtualizált környezetben az alaprendszert Host-nak nevezik, és a vendég rendszer kapja ezt az új módot.
- Ezzel ad a kernel szint alatt újabb CPU múködési módot, így támogatva a hardveres virtualizációt.
- Csak azért van Ring -1, mert más skálán nézzük.

83. Kérdés

*Mi a **POSIX** (Portable Operating System Interface for uniX) ? Van-e hatása a mai rendszerekben?*

- Egy könyvtári (API) ajánlás, hogy a különböző rendszerek nagyobb átjárhatósággal bírjanak. Ma szinte nagyobb a szerepe mint korábban.
- Igen van szerepe, de csak a Linux rendszereken, hiszen a Linux is valójában egyfajta Unix.
- Ez régi ajánlás, bizonyos konvenciók tartására, évről évre kisebb szereppel bír.
- Csökkent a szerepe, mert a korábbi támogatók közül a Microsoft nem tekinti feltétlenül irányadónak!

84. Kérdés

*Milyen elven működik az **optikai lemezen** az információ tárolása?*

- Az infravörös sugárzás elnyelésének elvén!
- Mágneses töltések elvén.
- A visszavert lézerfény időkülönbségén.
- Mint a hologram.

85. Kérdés

*Mi a klasszikus, legacy **BIOS** lemezkezelési módnak ma talán a legnagyobb hibája?*

- A legnagyobb fájlméret maximum 2GB.
- A fájlrendszer elemeinek száma korlátos.
- A maximális lemezméret ma már elég kicsi.
- Nincs hibája, az én számítógépem is ezt használja.

86. Kérdés

*Melyik **fájlrendszer** nem használ a **háttértár**ra érvényes helynyilvántartást?*

- Az NFS, mert úgyse tudjuk, hogy a távolban mi van.
- Mindegyik használ.
- A FAT nem használ ilyet.
- Csak a nagyon korai FAT16.

87. Kérdés

*Mi a **garantált ütemezés** lényege?*

- Garantálja az ütemező, hogy amikor CPU időt kér a folyamat, azt meg is kapja.
- A garantált ütemezés olyan prioritásos ütemezés, amiben a folyamat a prioritásával arányos CPU időt kap.
- Garantálja az ütemező, hogy az időintervallum lejártakor a következő folyamat jön.
- A prioritással fordított arányban osztjuk ki a CPU időket a folyamatok között.

88. Kérdés

*Mi a **szoftveres** és **hardveres megszakítás** közti különbség?*

- Nincs különbség.
- A sorrend, előbb mindig jön egy hardveres, aztán a szoftveres.
- A megszakítás kérés célja.
- A megszakítás kérés forrása.

89. Kérdés

*Egy **CPU** hogy tart **kapcsolat**ot egy külső eszközzel?*

- A CPU megszakítást küld az eszköznek, aminek hatására az küldi a következő adatot.
- Input-Output kapuk segítségével, melyek több környezetben memóriába vannak "ágyazva".

- A DMA eszköz segítségével.
- Megszakítások segítségével.

90. Kérdés

*Mi a **CFQ** (Completely Fair Queuing) blokkütemező jellemzője?*

- A strucc algoritmus használatát jelenti.
- A legrövidebb idő elvét követő algoritmus.
- Minden folyamat saját kéréssort kap, és ezen sorok között igazságosan el kell osztani a sávszélességet.
- Minden folyamat saját kéréssort kap, és ezen sorok között mindig a leghosszabb kapja a lehetőséget.

91. Kérdés

*Hol nem létezik a sorrendi ütemezés (**FCFS**) algoritmusa?*

- A lemezműveletek során csak az SSTF használt.
- Memóriakezelésnél nincs ilyen.
- Ezt mindenhol, még a boltban is használják.
- 1/0 eszközöknél.

92. Kérdés

*Melyik módszer nem **fájl elhelyezési stratégia**?*

- Az indextáblás elhelyezési stratégia.
- Láncolt tárolás elve.
- Folyamatos elhelyezési módszer.
- Az FCFS módszer szinte mint mindenhol itt is él. (az inkább olvasási stratégia)

93. Kérdés

*Mi nem feltétele a jó **kölcsönös kizárás**nak!*

- Nem lehet sebesség függő a jó kölcsönös kizárás.
- Nem lehet kívülről a benti folyamatot befolyásolni, blokkolni.
- Nem lehet CPU függő a jó kölcsönös kizárás.
- Ne legyen a folyamatoknak közös, osztott memóriája!

94. Kérdés

*Mi a probléma az alábbi **"Étkező filozófusok"** klasszikus probléma megoldásában?*

```
Void filozófus(int i)
{ while(1)
{
```

```
gondolkodom();
kell_villa(i); // bal villa
kell_villa((i+1)%N); //jobb
eszem();
nemkell_villa(i);
nemkell_villa((i+1)%N);
}
}
```

- Ha rosszkor vált az ütemező éhen halhatnak a filozófusok.
- Az a baj, hogy rosszul számolja a bal és jobboldali villa értékét.
- Azért rossz megoldás, mert a példában 5, páratlan volt a filozófusok száma.
- Ez jó megoldás, nincs holtpont.

95. Kérdés

*Jelölje meg mi nem jellemző a mai **Valós idejű folyamatok** ütemezésére!*

- Prioritási osztályokban FCFS vagy RR érvényesül.
- Nem prioritásos ütemező, mert a határidők számítanak csak.
- Prioritásos ütemező, a határidőket folyamat szinten kell tartani.
- Akkor ütemezhető, ha az egységnyi időre eső N esemény válaszidő összege <=1.

96. Kérdés

*A **memória kezelőnek** mikor nem kell a relokációval vagy a memória védelemmel foglalkozni?*

- Multi task környezetben ha láncolt listás memória nyilvántartást használunk.
- Ha monoprogramozási (sigle task) környezetben vagyunk.
- Ha multiprogramozási környezetben vagyunk és rögzített memória szeletek segítségével biztosítjuk a memóriát.
- Virtuális memória kezelésnél nem kell.

97. Kérdés

*Mit jelent a **titkosított fájlrendszer** fogalma?*

- A titkosítás azt biztosítja egy multi-user rendszerben, hogy a fájlrendszer tartalmát csak az adott felhasználó láthassa.
- A titkosítás azt biztosítja, hogy a fájlrendszer tartalmát kívülről egy független gépen ne lehessen olvasni.

- A titkosítás azt jelenti, hogy csak az adminisztrátorok látják azt a területet.
- A titkosítás azt biztosítja egy multi-user rendszerben, hogy a tulajdonában lévő állományokat, könyvtárakat csak az adott felhasználó láthassa.

98. Kérdés

*Mit nevezünk **szegmentált memória** elérésnek?*

- Ez egy egyindexes címzés, a laptábla szegmens indexű értéke megadja az offset értékét.
- Kétindexes címzés, az első index a virtuális laptábla, második index a valós cím, az eredmény lesz a memória tartalom.
- Ez egy virtuális címzés ami a virtuális gépek használata során jelentkezik.
- Mikor az egy memória cím használata helyett, egy címet két értékkel, egy szegmens és egy ezen belüli offset értékkel írunk le.

99. Kérdés

*Mikor kell **memória nyilvántartás**t vezetni a **memóriakezelő**nek?*

- Csak akkor ha nem rögzített memória szeletekkel dolgozik a memóriakezelő.
- Real-Time rendszerekben nem kell, minden másban igen.
- Csak ha bittérképes memória nyilvántartást használunk. v
- Gyakorlatilag akkor, ha nem monoprogramozási környezetben vagyunk.

100. Kérdés

*Mi a **RAID** (Redundant Array of Inexpensive Disks) rendszer szerepe?*

- Merevlemezek titkosítását segíti.
- Optikailemezek hatékony használatát segíti.
- Ez ma már nem használt az SSD diszkek kőrében.
- Adattárolás biztonságának növelése.

101. Kérdés

*Mit nevezünk **versenyhelyzet**nek a folyamatok mükődése kapcsán?*

- Mikor két vagy több folyamat közös adatot, kódot használ és a végeredmény függhet az elérés pillanatától.
- Mikor egy folyamat közös adatot, kódot használ és végeredményben függhet az elérés pillanatától.

- Amikor kiválasztjuk, hogy melyik folyamat jusson először erőforráshoz!
- Mikor két vagy több folyamat közös adatot, kódot használ és a végeredmény független mindentől.

102. Kérdés

*A kötegelt rendszerek **SJB**(Shortes Job first) folyamatütemező miért nem használható mai interaktív rendszerekben?*

- Mert az interaktív rendszerekben sok I/O igény jelentkezik.
- Mert a kötegelt rendszerek FCFS ütemezője a preferált.
- Használható lenne, de nem implementált.
- Mert ismerni kellene a futási időket előre.

103. Kérdés

*Melyik állítás igaz jó **folyamatütemezőr**e?*

- Páratlan, azonos elvek alapján osztja el a CPU-t a folyamatok között.
- A jelentkező igényeknek megfelelően osztja el a CPU-t, hasonlóan a bankár algoritmushoz.
- Nem érvényesítheti a prioritási elvet, mert ettől sérülhet a páratlanság.
- Nem érvényesítheti a prioritási elvet, mert ettől sérül az azonos elvek érvényesítésének szabálya.

104. Kérdés

*Melyik **program réteg** nem tartozik az eszköz és az operációs rendszer kapcsolatai közé?*

- Megszakításkezelés rétege.
- Eszköz független operációs rendszer interfész.
- Eszköz függő operációs rendszer interfész.
- Felhasználói I/O könyvtár.

105. Kérdés

*Mi az **eszközmeghajtó** program?*

- Olyan felhasználó szintű program, ami az alkalmazások és eszköz közvetlen kommunikációját segíti.
- Olyan speciális program, ami egy eszköz és az operációs rendszer közti kapcsolatot bíztosítja.
- Egy megszakítás kezelést végző task, ami az eszköz igényeire válaszol.

- Olyan kernel szintű program, ami az alkalmazások és az eszköz közvetlen kommunikációját segíti.

106.kérdés

Milyen BIOS lemezkezelési módot ismerünk a PC környezetben?

- Klasszikus MBR alapú es UEFI rendszerűt használhatunk.

107.kérdés

Ahogy a Gyártó-Fogyasztó probléma megoldásaként az alábbi algoritmusrészlet is mutatja, milyen funkciójú szemaforokról beszélhetünk? Stb.

- Kölcsönös kizárást és az alvás-ébredés kezelését biztosító szemaforokat.

108.kérdés

Hol használjuk a CFS (Completely Fair Scheduler) ütemezőt?

- A Linux rendszerek alapértelmezett ütemezője.

109.kérdés

Mit jelent a háttértár formázása? Szükséges ez?

Igen, a használatra kész helyzet beállítást végzi a fájlrendszernek megfelelően.

110.kérdés

Melyik állítás igaz egy jó folyamatütemezőre?

Pártatlan, azonos elvek alapján osztja el a CPU-t a folyamatok között.

111.kérdés

Mi a lapcserélési algoritmus feladata?

Ha nincs egy virtuális lap a memóriában, akkor egy lapot ki kell dobni a fizikai memóriából, és berakni helyére ezt az új lapot.

112.kérdés

Mit nevezünk fájlrendszernek?

Ez egy módszer, a fizikai lemezünkön, kötetünkön a fájlok és könyvtárak elhelyezési rendszerének kialakítására.